

PH12004/01 Python Programming Coursework

S2 April 2026

Candidate Number: XXXXX

Question 1 – Wien’s Displacement Law

Part (a) – Derivation

Starting from Planck’s radiation law:

$$\rho(\lambda) d\lambda = \frac{8\pi hc}{\lambda^5 (e^{hc/\lambda kT} - 1)} d\lambda \quad (1)$$

Making the substitution $x = hc/\lambda kT$, so that $\lambda = hc/xkT$, the two factors in equation (1) that depend on λ transform as:

$$\lambda^{-5} = \left(\frac{xkT}{hc}\right)^5 = \frac{x^5(kT)^5}{(hc)^5}, \quad e^{hc/\lambda kT} = e^x \quad (2)$$

Substituting both into equation (1) and collecting constants:

$$\rho(\lambda) = \frac{8\pi(kT)^5}{(hc)^4} \cdot \frac{x^5}{e^x - 1} \quad (3)$$

Since $x = hc/\lambda kT$ is a monotonically decreasing function of λ , maximising ρ with respect to λ is equivalent to maximising with respect to x . Absorbing all prefactors into the proportionality:

$$\rho \propto \frac{x^5}{e^x - 1} \quad (4)$$

Differentiating with respect to x and setting equal to zero:

$$\frac{d\rho}{dx} \propto \frac{5x^4(e^x - 1) - x^5 e^x}{(e^x - 1)^2} = 0 \quad (5)$$

Numerator must equal zero as the denominator is non-zero. Dividing through by x^4 :

$$5(e^x - 1) - x e^x = 0 \quad (6)$$

which rearranges to:

$$\boxed{e^x(5 - x) - 5 = 0} \quad (7)$$

Once the root x^* of equation (7) is found, Wien’s displacement law follows from the substitution made at the beginning:

$$\lambda_{\max} T = \frac{hc}{x^* k} = b \quad (8)$$

where b is Wien’s displacement constant.

Part (b) – Newton-Raphson Solution

The equation $f(x) = e^x(5 - x) - 5 = 0$ was solved using the Newton-Raphson method, with a central-difference numerical derivative:

$$f'(x) \approx \frac{f(x + h/2) - f(x - h/2)}{h}, \quad h = 10^{-5} \quad (9)$$

$\mathbf{f(x)}$ is $e^x(5 - x) - 5$, it's defined globally so it can be called throughout the code without having to repeat it. $\mathbf{df(x)}$ implements the central difference formula using two calls to $\mathbf{f(x)}$ on either side of the evaluation point with `step = 1e-5`.

The main loop uses `while True` with a `break` condition instead of a `for` loop with a fixed iteration count, which is what was originally tried, because Newton-Raphson doesn't always take the same number of steps to converge. $x_0 = 5$ was chosen as an upper bound because when x is greater than 5, the term $(5 - x)$ becomes negative which means that $e^x(5 - x) - 5$ is always negative, so the root must lie below $x = 5$. The `iteration` variable was added to see how many iterations were needed for it to converge.

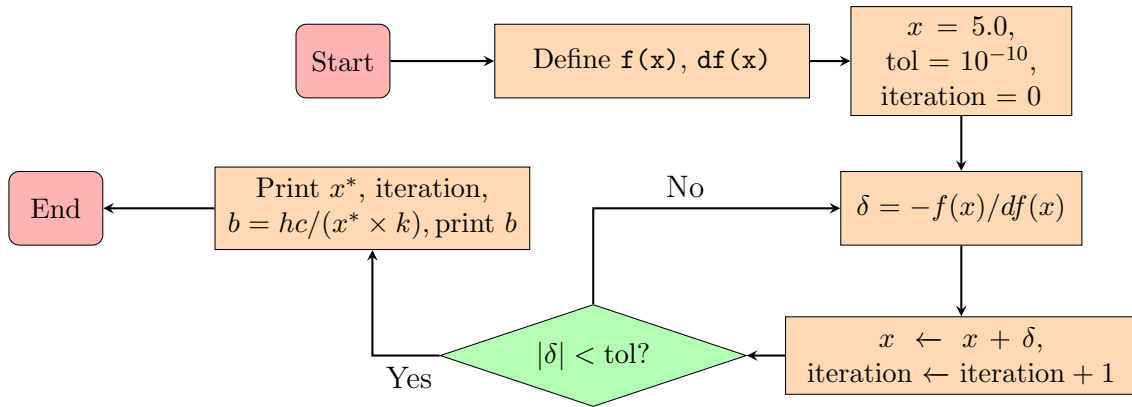


Figure 1: Flowchart for the Newton-Raphson solution (Q1b).

The script converged in 4 iterations, giving:

$$x^* = 4.96511423 \quad (10)$$

$$b = \frac{hc}{x^*k} = 2.898 \times 10^{-3} \text{ m K} \quad (11)$$

This agrees with the known value of Wien's Constant: $2.898 \times 10^{-3} \text{ m K}$.

Question 2 – Telescope Diffraction Pattern

Part (a) – Bessel Functions

The Bessel functions $J_m(x)$ are defined by:

$$J_m(x) = \frac{1}{\pi} \int_0^\pi \cos(m\theta - x \sin \theta) d\theta \quad (12)$$

This integral was evaluated using the extended trapezium rule with $N = 10000$ strips. `J(m, x, N=10000)` takes the order `m` and point `x` as required arguments, with `N` as a keyword argument set to 10000. The trapezium rule needs the end terms halved so $\theta = 0$

and $\theta = \pi$ were handled separately, then the rest of the terms are summed in a for loop. `theta = i * h` was used instead of simply summing at each step because of floating point accumulation, though for $N = 10000$ this might not have been necessary.

Two checks were performed before plotting: $J_0(0) = 1.000000$ and $J_1(0) = 0.000000$, both were confirmed to six d.p. The three list comprehensions `[J(0, x) for x in x_vals]` evaluate the function at all 200 x-values before plotting.

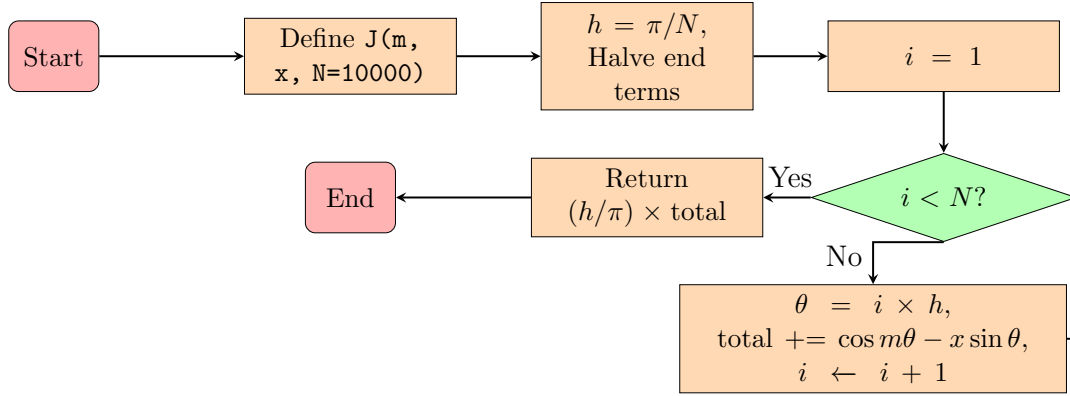


Figure 2: Flowchart for the Bessel function $J(m, x)$ (Q2a). End terms at $\theta = 0$ and $\theta = \pi$ are halved separately.

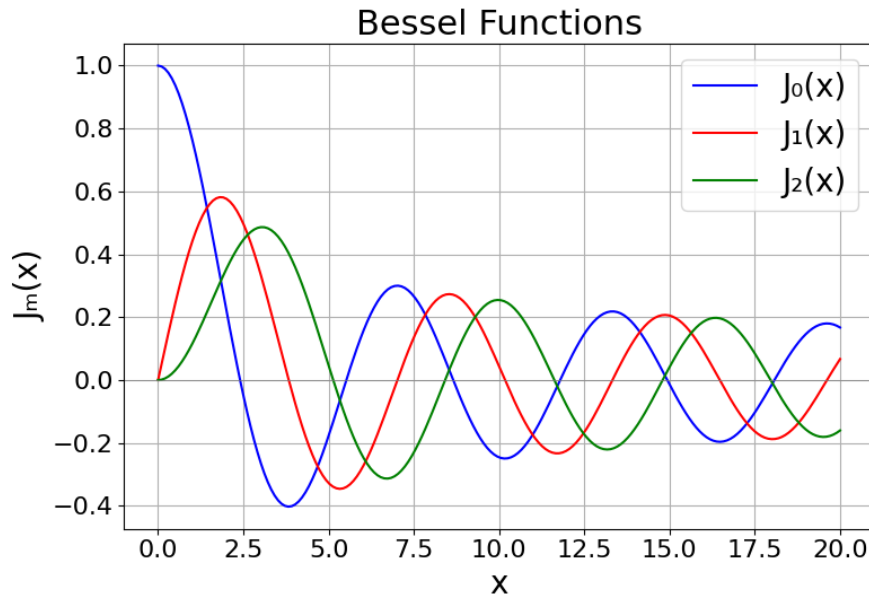


Figure 3: Bessel functions $J_0(x)$, $J_1(x)$ and $J_2(x)$ for $x = 0$ to 20, calculated using the trapezium rule with $N = 10000$.

The functions show oscillation with decreasing amplitude, with $J_0(0) = 1$ and $J_m(0) = 0$ for $m > 0$. The plot shows the first zero of $J_0(x)$ at $x \approx 2.40$ and the first zero of $J_1(x)$ at $x \approx 3.83$, consistent with the known values $x_{0,1} = 2.4048$ and $x_{1,1} = 3.8317$ to four significant figures [1]. The decreasing amplitude with x is consistent with the large- x asymptotic form $J_m(x) \sim \sqrt{2/\pi x} \cos(x - m\pi/2 - \pi/4)$ [1].

Part (b) – Airy Diffraction Pattern

The intensity of the diffraction pattern is given by:

$$I(r) = I_0 \left(\frac{2J_1(x)}{x} \right)^2, \quad x = \frac{2\pi}{\lambda} a \frac{r}{R} \quad (13)$$

For a focal ratio $R/2a = 10$, this simplifies to:

$$x = \frac{\pi r}{10\lambda} \quad (14)$$

A wavelength of $\lambda = 550 \text{ nm}$ was used as it's in the middle of the visible range of light. $J(m, x, N=10000)$ is copied from Q2(a). `r_vals` uses `np.linspace` with 300 points over $\pm 25 \mu\text{m}$. 300 points was chosen to ensure a smooth curve while keeping the runtime reasonable as every call to `J` requires 10000 iterations. The intensity is built up by appending to `I_vals` in a for loop rather than a list comprehension, because the $r = 0$ case requires an `if` statement. At $r = 0$, the formula $2J_1(x)/x$ would cause division by zero, so this is handled by checking `abs(r) < 1e-12` and setting $I = I_0$ directly, using the known analytical limit $\lim_{x \rightarrow 0} 2J_1(x)/x = 1$. The absolute value of r is used when computing x , since the diffraction pattern is symmetric about $r = 0$.

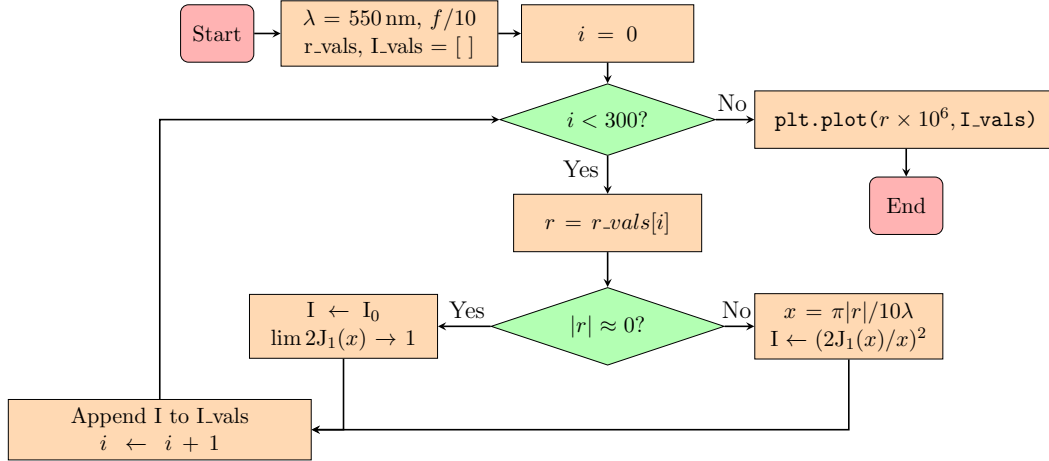


Figure 4: Flowchart for the Airy diffraction pattern calculation (Q2b). The special $r = 0$ case is dealt with on its own before it's appended to `I_vals`.

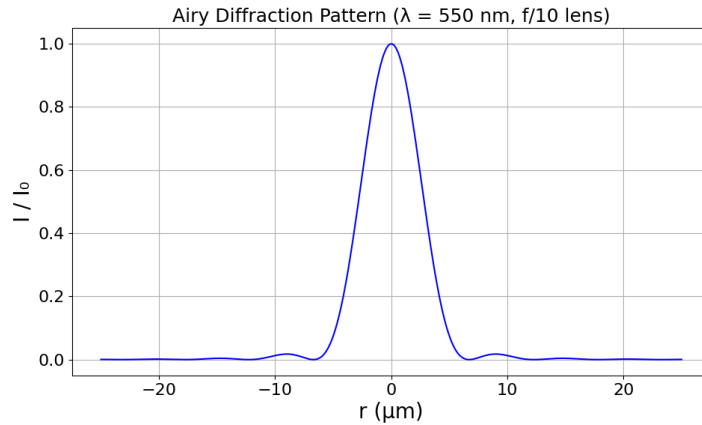


Figure 5: Airy diffraction pattern for a circular lens with focal ratio $f/10$ and $\lambda = 550 \text{ nm}$. The central bright Airy disk is clearly visible, surrounded by much fainter rings.

The first dark ring occurs at the first zero of $J_1(x)$, at $x \approx 3.83$, corresponding to:

$$r_{\min} = \frac{3.83 \times 10 \times \lambda}{\pi} \approx 6.7 \mu\text{m} \quad (15)$$

The radius of the first dark ring $r_{\min} \approx 6.7 \mu\text{m}$ corresponds to the Airy disk radius. By the Rayleigh criterion, two point sources are just resolvable when the central maximum of one falls on the first minimum of the other, giving a minimum resolvable angular separation of $\Delta\theta = 1.22\lambda/D$ [2]. A larger aperture reduces $r_{\min}$ and improves resolution, which is why astronomical telescopes use large mirrors.

References

- [1] Jackson, J.D., 1999. *Classical electrodynamics*. 3rd ed. New York: Wiley, p. 114.
- [2] Hecht, E., 2017. *Optics*. 5th ed. Harlow: Pearson, pp. 490–493.

Appendix – Python Code

Q1(b) – Wien’s Law

```
import numpy as np

# constants
h = 6.626e-34
c = 2.998e8
k = 1.3806e-23

# f(x) = e^x * (5-x) - 5
def f(x):
    return np.exp(x) * (5 - x) - 5

# central difference derivative of f
def df(x):
    step = 1e-5
    return (f(x + step/2) - f(x - step/2)) / step

x = 5.0
tolerance = 1e-10
iteration = 0

# Newton-Raphson loop
while True:
    delta = -f(x) / df(x)
    x = x + delta
    iteration += 1
    if abs(delta) < tolerance:
        break
```

```

print(f"x = {x:.8f}")
print(f"Iterations = {iteration}")

# Wien's constant
b = h * c / (x * k)
print(f"Wien's constant b = {b:.3e} m.K")

```

Q2(a) – Bessel Functions

```

import numpy as np
import matplotlib.pyplot as plt

# Bessel function J_m(x) via trapezium rule
def J(m, x, N=10000):
    h = np.pi / N
    # end terms
    total = 0.5 * (np.cos(m*0 - x*np.sin(0)) + np.cos(m*np.pi - x*np.sin(np.pi)))
    # interior terms
    for i in range(1, N):
        theta = i * h
        total += np.cos(m * theta - x * np.sin(theta))
    return (h / np.pi) * total

# checks
print(f"J_0(0) = {J(0,0):.6f} (should be 1.0)")
print(f"J_1(0) = {J(1,0):.6f} (should be 0.0)")

x_vals = np.linspace(0, 20, 200)

# evaluate J0, J1, J2 at all x values
J0_vals = [J(0, x) for x in x_vals]
J1_vals = [J(1, x) for x in x_vals]
J2_vals = [J(2, x) for x in x_vals]

plt.figure(figsize=(8,5))
plt.plot(x_vals, J0_vals, 'b-', label='J_0(x)')
plt.plot(x_vals, J1_vals, 'r-', label='J_1(x)')
plt.plot(x_vals, J2_vals, 'g-', label='J_2(x)')
plt.axhline(0, color='gray', lw=0.5)
plt.xlabel('x', fontsize=20)
plt.ylabel('J_m(x)', fontsize=20)
plt.title('Bessel Functions', fontsize=22)
plt.legend(fontsize=20)
plt.grid(True)
plt.tick_params(axis='both', labelsize=16)
plt.show()

```

Q2(b) – Diffraction Pattern

```
import numpy as np
import matplotlib.pyplot as plt

# Bessel function
def J(m, x, N=10000):
    h = np.pi / N
    total = 0.5 * (np.cos(m*0 - x*np.sin(0)) + np.cos(m*np.pi - x*np.sin(np.pi)))
    for i in range(1, N):
        theta = i * h
        total += np.cos(m * theta - x * np.sin(theta))
    return (h / np.pi) * total

lam = 550e-9          # metres
focal_ratio = 10

r_vals = np.linspace(-25e-6, 25e-6, 300)    # metres
I_vals = []

for r in r_vals:
    if abs(r) < 1e-12:    # r=0 case
        I_vals.append(1.0)
    else:
        x = np.pi * abs(r) / (focal_ratio * lam)
        val = 2 * J(1, x) / x
        I_vals.append(val**2)

plt.figure(figsize=(8,5))
plt.plot(r_vals * 1e6, I_vals, 'b-')    # convert r to microns
plt.xlabel('r (μm)', fontsize=20)
plt.ylabel('I / I0', fontsize=20)
plt.title('Airy Diffraction Pattern (λ = 550 nm, f/10 lens)', fontsize=18)
plt.grid(True)
plt.tick_params(axis='both', labelsize=16)
plt.show()
```